Event-driven dynamics

Simon Frost

Table of contents

1 Overview 1

2 Rules as rates 2

3 One event at a time 3

4 Summaries over time 3

5 Next-reaction and direct methods 4

6 Interventions at exact times 4

7 A sparse, seasonal landscape 6

8 Ensembles 7

9 The same rule in discrete time 8

1 Overview

Many plant-disease and pest models are written as rates, not as per-step probabilities: a susceptible plant becomes infected at rate β times its number of infected neighbours, and an infected plant is removed at rate γ. Such a model is a continuous-time Markov chain on the lattice. This vignette shows how to write one as an `EventRule`, simulate it exactly, event by event, with the `Gillespie` scheme, act on it at exact times, and run the same rule in discrete time.

```
ENV["GKSwstype"] = "100"
using CellularAutomataCore
using Plots
using Random
using Statistics
```

2 Rules as rates

An `EventRule`'s function returns the cell's possible transitions as `rate ⇒ new_state` pairs. Here is the contact process (an SIS epidemic): susceptible cells (0) are infected by each infected neighbour at rate β, and infected cells (1) recover at rate γ.

```
sis = EventRule(VonNeumann(1)) do cell, nbrs, ctx
    if cell == 0
        (ctx.p.β * count(==(1), nbrs) ⇒ 1,)
    else
        (ctx.p.γ ⇒ 0,)
    end
end
```

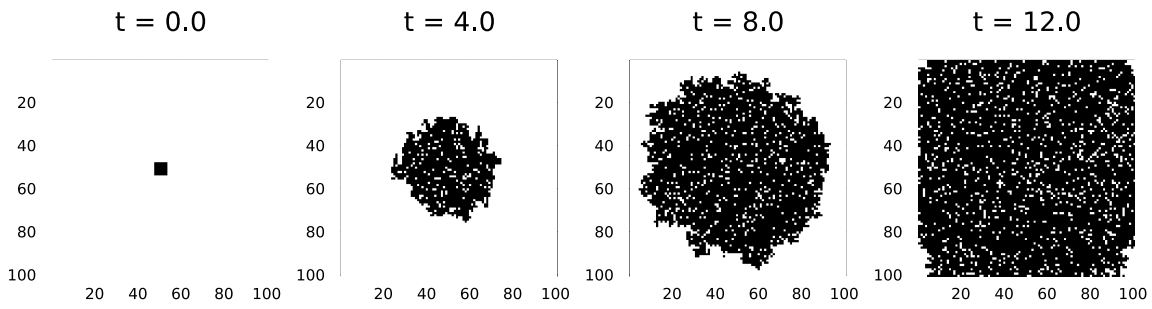
```
EventRule(<anonymous function>, VonNeumann(1))
```

A problem with an event rule takes real times. `Gillespie()` simulates it exactly, and `saveat` takes real saving times.

```
n = 100
u0 = zeros(Int8, n, n)
u0[48:53, 48:53] .= 1
prob = CAProblem(sis, u0, 12.0, (β=2.5, γ=1.0); seed=1)
sol = solve(prob, Gillespie(); saveat=4.0)
sol.t
```

```
4-element Vector{Float64}:
  0.0
  4.0
  8.0
 12.0
```

```
plot([plot(sol; snapshot=i, c=:binary, colorbar=false, title="t = $(sol.t[i])")
      for i in eachindex(sol.t)]...; layout=(1, 4), size=(900, 250))
```



The contact process on the square lattice persists only when β/γ exceeds about 1.65. At 2.5 the infection spreads outwards as a ragged front, and recoveries leave holes behind it that are soon reinfected.

3 One event at a time

A step of a continuous-time integrator is one event: the time jumps to the next event and exactly one cell changes.

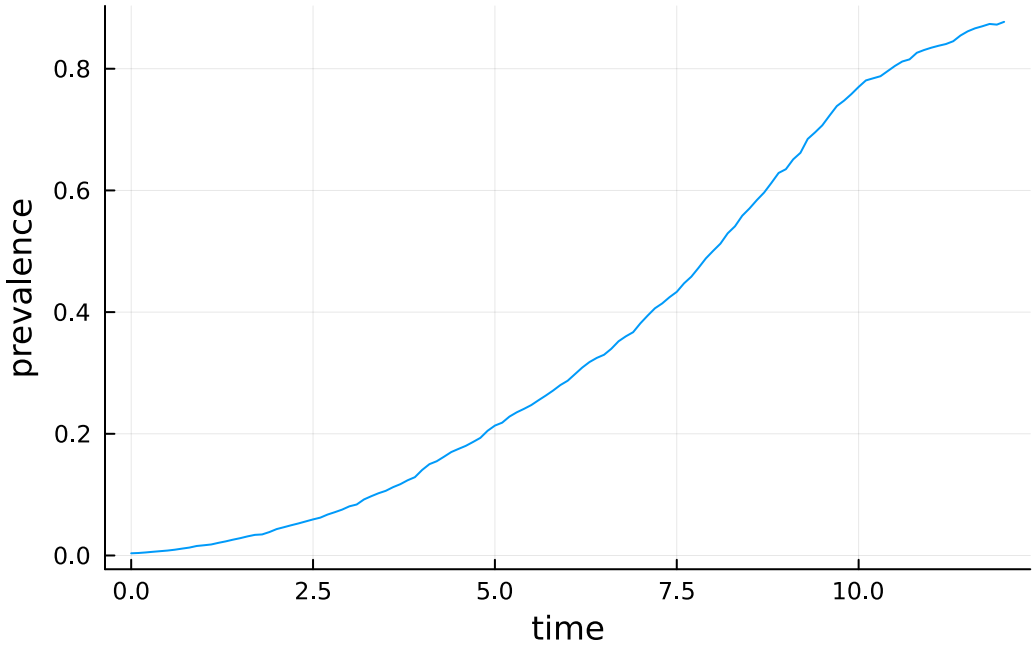
```
integ = init(prob, Gillespie())
before = copy(integ.u)
step!(integ)
integ.t, findall(integ.u .!= before)
```

```
(0.006665324843065025, CartesianIndex{2}[CartesianIndex(54, 51)])
```

4 Summaries over time

Saving whole lattices is expensive. A `SavingCallback` records a summary at real times instead; here the prevalence every 0.1 time units.

```
prevalence = SavedValues(Float64, Float64)
solve(prob, Gillespie(); callback=SavingCallback((u, t, integ) -> mean(u),
prevalence;
                                                 saveat=0.1))
plot(prevalence.t, prevalence.saveval; xlabel="time", ylabel="prevalence",
legend=false)
```



5 Next-reaction and direct methods

`Gillespie(:next_reaction)`, the default, gives every cell its own exponential clock in a heap. After an event it recomputes only the cell and its neighbours, so an event costs the same however large the lattice is. `Gillespie(:direct)` sums every rate at every event; it is simple and serves as a reference. The two sample the same process but use their random numbers differently, so they agree in distribution, not path by path.

```
function final_prevalences(method; reps=40)
    [mean(final_state(solve(remake(prob; tspan=3.0, seed=s), Gillespie(method))))
     for s in 1:reps]
end
nr, dm = final_prevalences(:next_reaction), final_prevalences(:direct)
(next_reaction=(mean(nr), std(nr)), direct=(mean(dm), std(dm)))
```

```
(next_reaction = (0.0823575, 0.008374898009329229), direct = (0.08184750000000002,
0.00795738610284455))
```

```
small = remake(prob; tspan=3.0)
solve(small, Gillespie(:next_reaction)); solve(small, Gillespie(:direct))
t_nr = @elapsed solve(small, Gillespie(:next_reaction))
t_dm = @elapsed solve(small, Gillespie(:direct))
(next_reaction=t_nr, direct=t_dm, ratio=t_dm / t_nr)
```

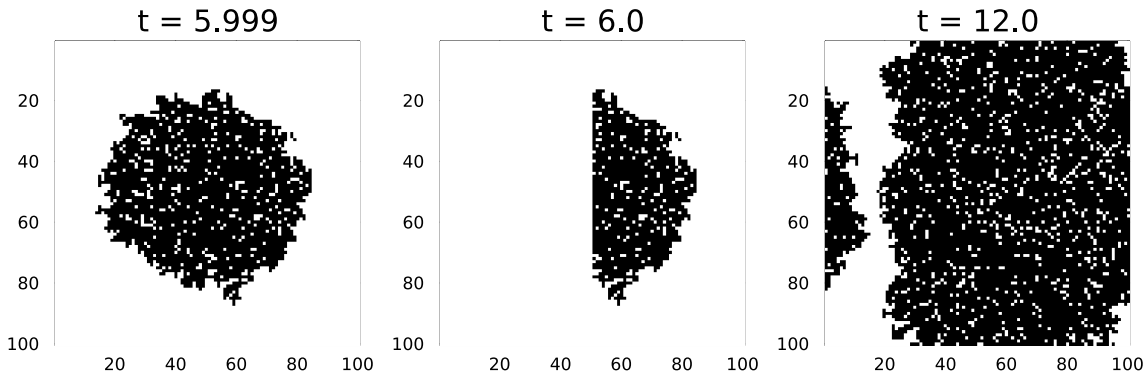
```
(next_reaction = 0.002414042, direct = 0.032164209, ratio = 13.323798426042298)
```

The random numbers are counter-based, as everywhere in the package: a cell's n-th waiting time is a function of the seed, the cell and n alone. A run is reproducible and does not depend on the order in which events are processed.

6 Interventions at exact times

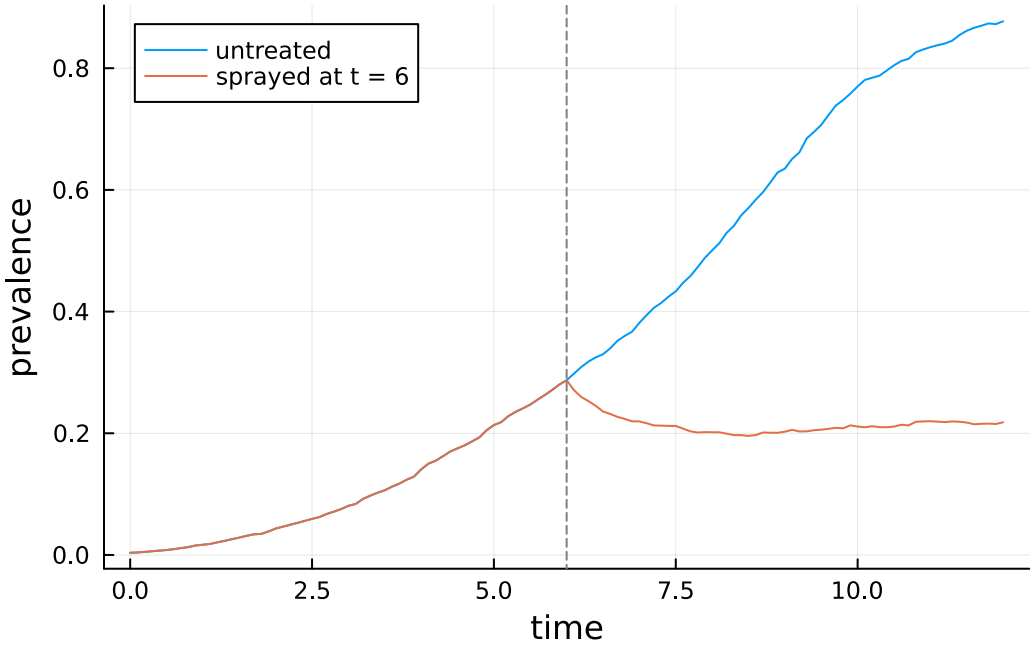
Callbacks work as in discrete time. `PresetTimeCallback` and `PeriodicCallback` also make the simulation stop at exactly their times, even when no event falls there. Here every infected cell in the left half of the lattice is cleared (roguing) at $t = 6$.

```
rogue! = integ -> (integ.u[:, 1:(n ÷ 2)] .= 0)
s = solve(prob, Gillespie(); saveat=[5.999, 6.0, 12.0],
          callback=PresetTimeCallback([6.0], rogue!))
plot([plot(s; snapshot=i, c=:binary, colorbar=false, title="t = $(s.t[i])")
      for i in 2:4]...; layout=(1, 3), size=(800, 260))
```



Changing the parameters from a callback (`integ.p = ...`) changes every rate from that moment on. Here transmission falls to a quarter at $t = 6$, as if a protectant were sprayed. The prevalence drops at once and then holds roughly steady. $\beta/\gamma = 0.625$ is below the lattice threshold, so the infection will die out, but slowly: each cell still has four neighbours to infect, and dense infected patches persist for a long time before fluctuations clear them.

```
spray = PresetTimeCallback([6.0], integ -> (integ.p = (β=integ.p.β / 4,
γ=integ.p.γ)))
sprayed = SavedValues(Float64, Float64)
solve(prob, Gillespie();
      callback=CallbackSet(spray, SavingCallback((u, t, i) -> mean(u), sprayed;
saveat=0.1)))
plot(prevalence.t, prevalence.saveval; label="untreated", xlabel="time",
     ylabel="prevalence")
plot!(sprayed.t, sprayed.saveval; label="sprayed at t = 6")
vline!([6.0]; label=nothing, ls=:dash, c=:grey)
```



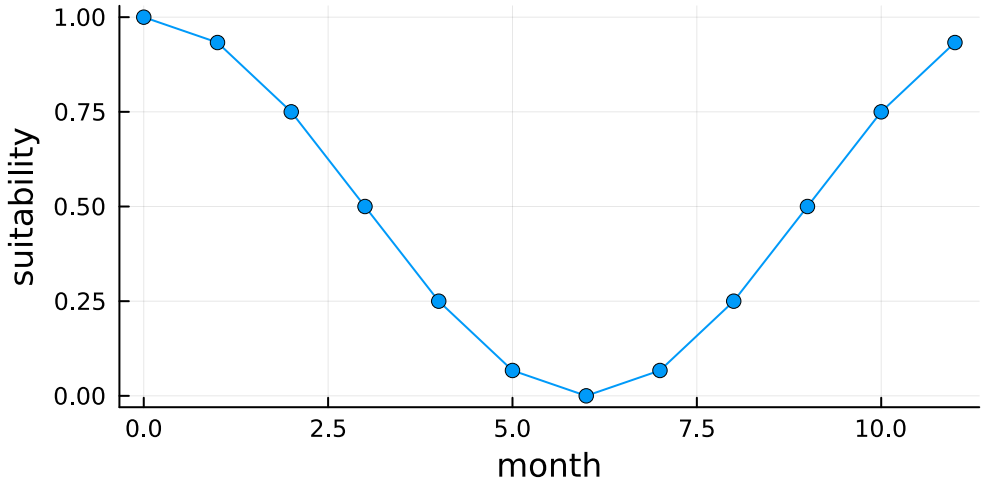
7 A sparse, seasonal landscape

Real host landscapes are patchy and the weather is seasonal. This model is an SIR epidemic among plants that occupy 60% of the cells, written with `@camodel` and an `@events` block. Cells without a host are masked, so they never change. Infection is driven by a seasonal suitability covariate with one slice per unit of time. In continuous time the rule sees `ctx.t`, the integer part of the time, and the rates are recomputed at every whole time.

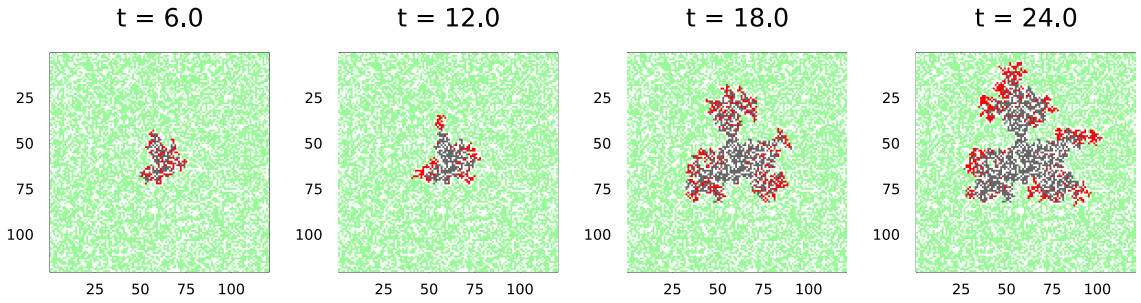
```
sir = @camodel PlantSIR begin
    @description "Plant SIR epidemic with seasonal transmission"
    @states state::Int8 = 0 ∈ (0, 1, 2, 3)   # 0 no host, 1 S, 2 I, 3 R
    @parameters β = 1.2 γ = 0.25
    @neighbourhood Moore(1)
    @covariates season
    @events begin
        if cell == 1
            (β * covariate(ctx, :season) * count(==(2), nbrs) ⇒ Int8(2),)
        elseif cell == 2
            (γ ⇒ Int8(3),)
        else
            ()
        end
    end
end
```

```
CAModel PlantSIR
  Plant SIR epidemic with seasonal transmission
  states:
    state::Int8 = 0 ∈ (0, 1, 2, 3)
  parameters:
    β = 1.2
    γ = 0.25
  covariates: season
  rule: EventRule(<anonymous function>, Moore(1))
```

```
m = 120
hosts = rand(Xoshiro(3), m, m) .< 0.6
start = Int8.(hosts)              # every host susceptible ...
for I in CartesianIndices((58:62, 58:62))
    hosts[I] && (start[I] = 2)    # ... except a small infected patch
end
season = cat([fill(0.5 + 0.5 * cospi(2t / 12), m, m) for t in 0:11]...; dims=3)
plot(0:11, season[1, 1, :]; xlabel="month", ylabel="suitability", legend=false,
     size=(500, 250), marker=:circle)
```



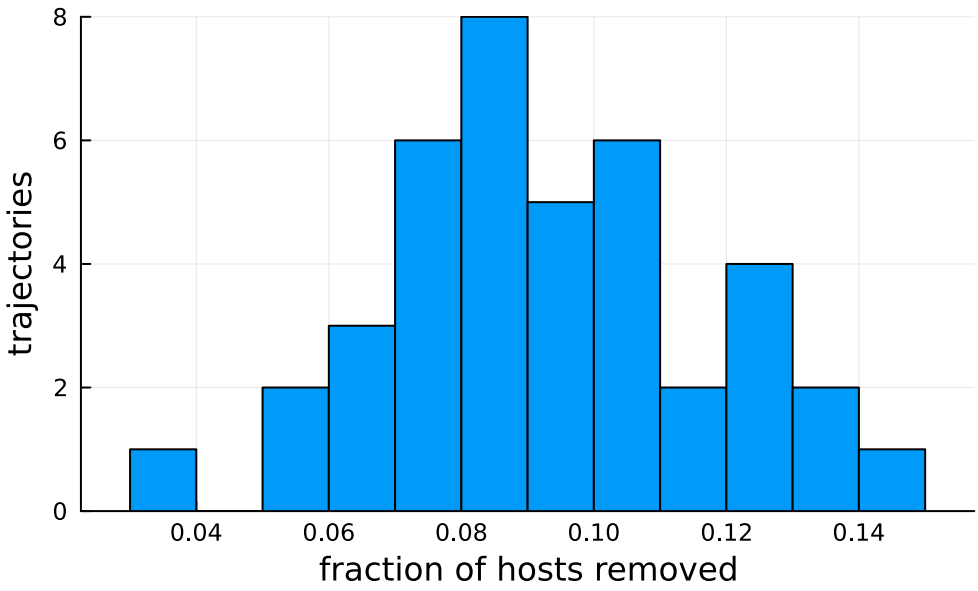

```
p = CAProblem(sir, (state=start,), 24.0; space=Torus(m, m), mask=hosts,
              covariates=(season=season,), seed=4)
ssol = solve(p, Gillespie(); saveat=6.0)
plot([plot(Torus(m, m), u; c=[:white, :palegreen, :red, :grey40], clims=(0, 3),
           colorbar=false, title="t = $t") for (t, u) in zip(ssol.t[2:end],
ssol.u[2:end])]...;
     layout=(1, 4), size=(950, 260))
```



8 Ensembles

An `EnsembleProblem` gives every trajectory its own seed, as in discrete time. Here is the distribution of the final epidemic size in the seasonal landscape.

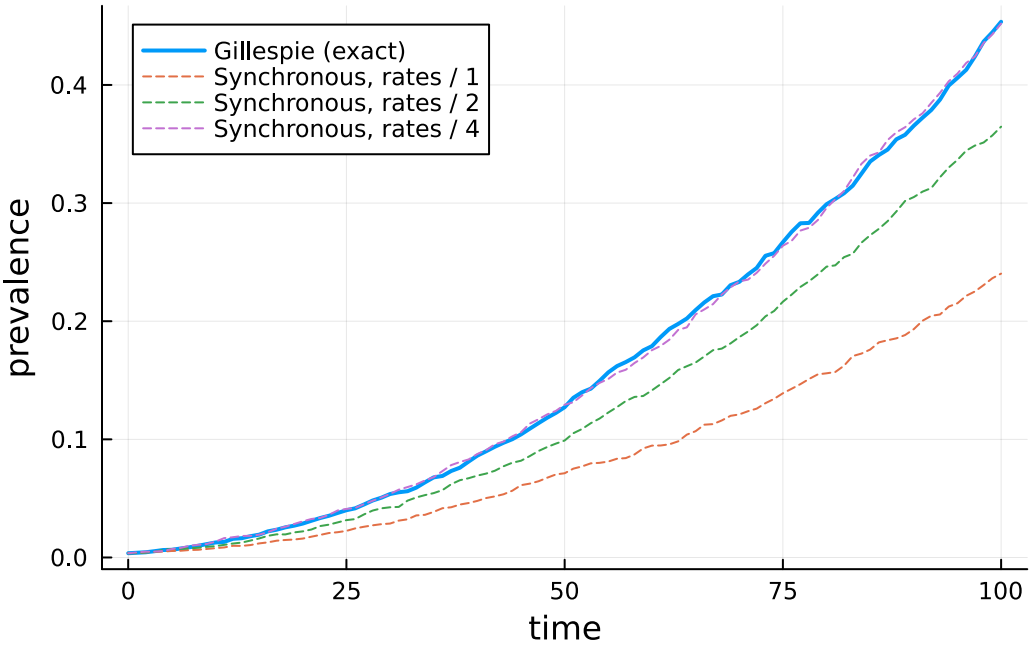
```
ens = solve(EnsembleProblem(p; output_func=(s, ctx) -> (count(==(3),
final_state(s)), false)),
            Gillespie(), EnsembleThreads(); trajectories=40, save_everystep=false)
histogram(ens.u ./ count(hosts); bins=15, xlabel="fraction of hosts removed",
          ylabel="trajectories", legend=false, size=(500, 300))
```



9 The same rule in discrete time

An event rule also runs under the synchronous schemes. In each step a cell with total rate a changes with probability `1 - exp(-a)`, and if it changes, it picks a transition in proportion to the rates. This is the usual chain-binomial discretisation, and it runs on every backend, GPUs included. It is an approximation: a cell infected during a step cannot pass the infection on until the next one, so the discrete-time epidemic lags the exact one. The lag shrinks as the step shrinks relative to the rates. Below, the rates are divided by `k` and the run lengthened `k` times, which leaves the exact process unchanged up to a rescaling of time but makes each step a smaller slice of it.

```
function prevalence_curve(k; exact)
    q = remake(prob; p=(β=0.2 / k, γ=0.1 / k), tspan=exact ? 100.0k : 100k)
    saved = SavedValues(exact ? Float64 : Int, Float64)
    cb = SavingCallback((u, t, i) -> mean(u), saved; saveat=k)
    exact ? solve(q, Gillespie(); callback=cb) : solve(q; callback=cb,
save_everystep=false)
    return saved.t ./ k, saved.saveval
end
plot(prevalence_curve(1; exact=true)...; label="Gillespie (exact)", lw=2,
     xlabel="time", ylabel="prevalence", legend=:topleft)
for k in (1, 2, 4)
    plot!(prevalence_curve(k; exact=false)...; label="Synchronous, rates / $k",
    ls=:dash)
end
plot!()
```


Gillespie (exact)
Synchronous, rates / 1
Synchronous, rates / 2
Synchronous, rates / 4
prevalence
0.0
0.1
0.2
0.3
0.4
0
25
50
75
100
time